MMAUG / AI & DEVOPS BOOTCAMP

Agent Chaos to Agent Control

Building an Enterprise Agent Governance Layer with Microsoft Foundry

Scan to join the Livestream

Rajaniesh Kaushikk / Microsoft MVP & Databricks MVP

[Add session date]

A Global Community for AI, Cloud, Infra, DevOps and Tech Enthusiasts

Learn. Innovate. Build.

mmaug.com

Meet the speaker

25 years in enterprise data and AI, most of it in regulated industries

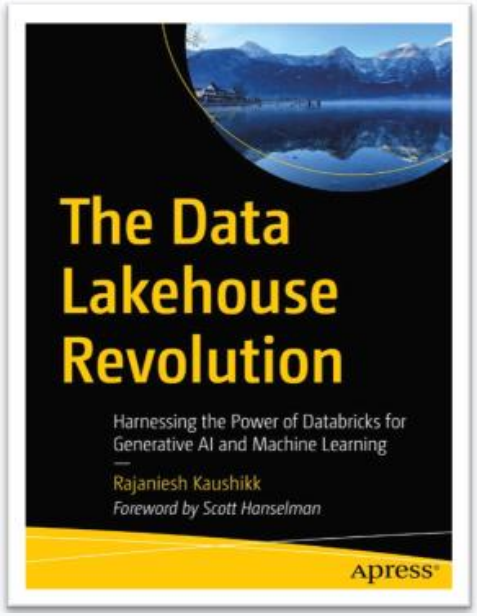


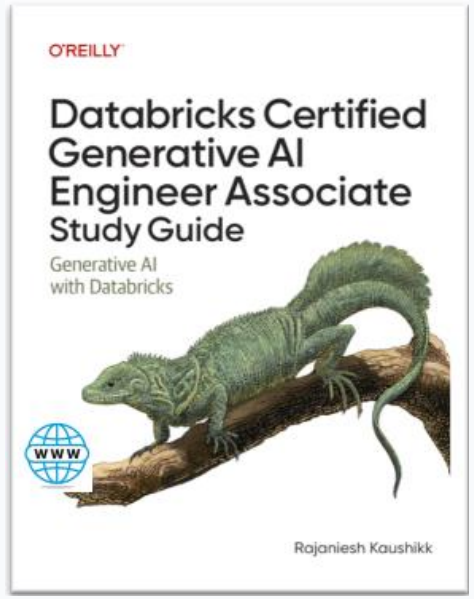
Rajaniesh Kaushikk

Director of Technology, Virtusa

Microsoft® Most Valuable Professional

databricks MVP





I build enterprise data and AI platforms for pharma, life sciences and banking clients. These are regulated environments, where adding governance later is not an option.

- Dual Microsoft MVP and Databricks MVP, Databricks Champion, Microsoft Certified Trainer
- Author of two books on Databricks and the data lakehouse (O'Reilly, Apress)
- I have watched agent governance become the problem nobody budgeted for

rajanieshkaushikk.com **youtube.com/c/RajaniesKaushikk**

CONNECT

Today’s session

Where we are going in the next 45 minutes

01	**The problem**	What breaks after you launch your first twenty agents
02	**The layer**	Five controls between every agent and every system
03	**Foundry**	What the platform does for you, and what you still own
04	**Live demo**	From chaos to control, in four acts
05	**Monday**	Five things you can start when you get back

SECTION / 01

WHO APPROVED THAT TOOL CALL?

> *Twenty agents, five business units, no list of what any of them can do.*

Four failures that appear after launch

None of these show up in the demo. All of them show up in production.

01

01

Credential sprawl

Every agent carries its own API keys, copied across repos and rarely rotated.

02

02

Rogue tool calls

An agent invokes a write action it was never meant to have, because nothing checked first.

03

03

No audit trail

Compliance asks which agent touched a record. Nobody can answer from the logs.

04

04

No visibility

Five business units, five agent stacks, no shared inventory of agents, owners or tools.

Each one maps to a control we build in the next section.

The merge nobody could trace

A composite from regulated-industry work. Not one client, and the details are invented.

WHAT HAPPENED

- A pharmacovigilance team built an agent to summarise adverse-event cases.
- During a backlog push, someone asked it to tidy up duplicates.
- It merged 40 cases and overwrote the narrative field on 12 of them.
- Nobody noticed for three weeks.

WHY IT HAPPENED

- Not because the model misbehaved.
- To ship fast, the team reused the service principal from an old ETL job.
- That principal already had write access to the safety database.
- The audit log showed the ETL account, not the agent.

CAUGHT BY **An identity scoped to read, a human approval on any write, and a trace naming the agent.**

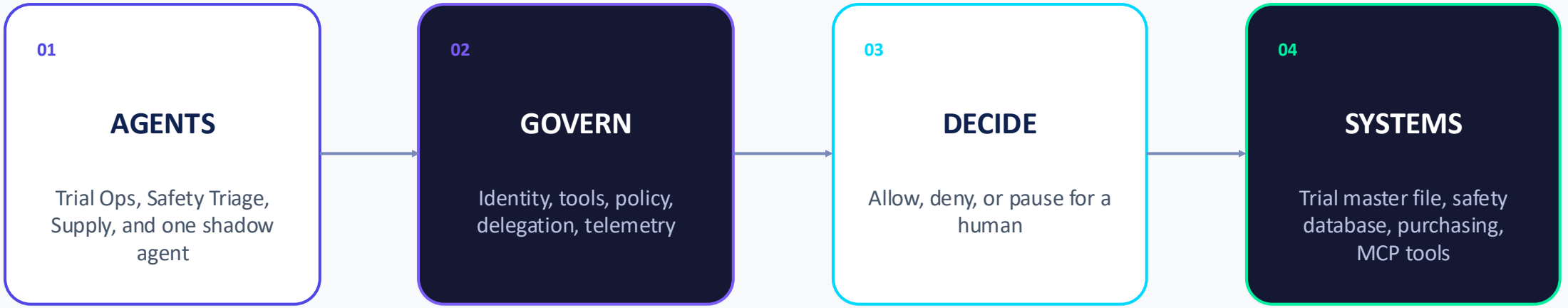
SECTION / 02

THE GOVERNANCE LAYER

> *Every agent call passes one place, and answers five questions.*

One layer between every agent and system

Agents never touch systems directly



Every call is recorded: the agent, the tool, the decision and the rule that fired

Every agent gets its own identity

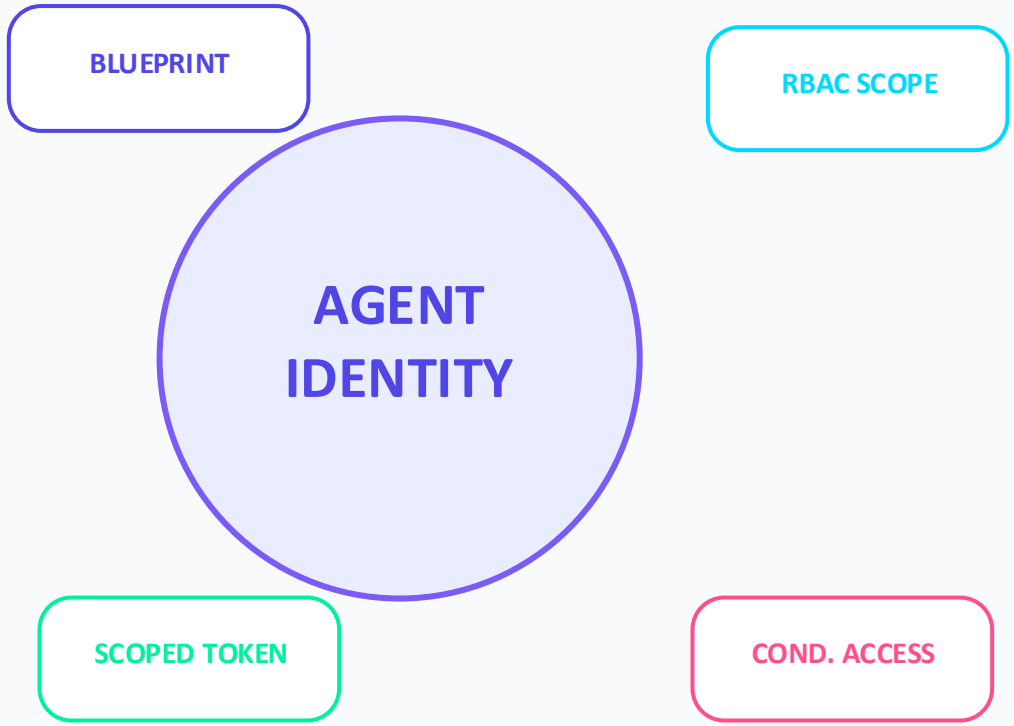
Credential sprawl starts because agents borrow keys instead of having identities

One identity per agent

In Entra Agent ID a blueprint is the template that holds credentials and policies. Each agent identity it creates is a real principal in Entra with no secret of its own.

WATCH OUT

An API key never reaches Entra, so Conditional Access cannot see it.



One governed endpoint for every tool

Foundry Toolboxes, in public preview since April 2026

BEFORE

- Three agents, five tools, fifteen separate connections.
- Every line has its own config and its own credential.
- Adding a tool means touching every agent that needs it.
- Now scale that to twenty agents.

AFTER

- One MCP-compatible endpoint for the whole bundle.
- Agents discover tools at runtime, with no redeploy.
- The toolbox injects credentials and refreshes tokens.
- Versioned: test a version, then promote it to default.

NOT LOCK-IN **Any MCP client can consume it: Agent Framework, LangGraph, even Claude Code.**

Deny by default, per agent and per tool

This is the policy file from the demo, not a diagram

policy.yaml

```
01 default: deny
02 trial_ops:      search_docs allow
03 safety_triage:  read_case allow, update_case approve
04 supply:         search_docs allow, create_po allow
05 shadow_agent:   unregistered, deny everything
06 approve = a human confirms before the call runs
```

1. **Trial Ops never needs patient case data, so it never gets it.**
2. **Approve means a human confirms before any write to a safety case.**
3. **Three gates enforce it: the toolbox, the Entra token, and policy middleware.**

A checkpoint for agent-to-agent calls

MCP governs agent to tool. A2A is where data leaves your boundary.

01

01

Allow-list

Is the remote agent's card on the approved list?

02

02

Identity

May this agent identity delegate to that one?

03

03

Depth

Is the delegation chain still under the hop limit?

04

04

Data

Does the payload's classification allow it to leave?

A denied delegation is traced too. It becomes an audit event, not silence.

One trace, prompt to last tool call

Agent Framework emits OpenTelemetry using the GenAI semantic conventions

```
trace 4f2a9c

01 invoke_agent trial_ops              1.84s
02   execute_tool search_docs   allow   0.31s
03   invoke_agent safety_triage        0.92s
04     governance.policy DENY rule_14
05     execute_tool update_case blocked   --
06 entra.agent_id + governance.decision on every span
```

1. **The same trace ID carries across the agent-to-agent hop.**
2. **The denial is a span. A blocked call leaves evidence, not silence.**
3. **Which agent touched this record, and why, becomes a trace query.**

SECTION / 03

LIVE DEMO

> *Four acts, synthetic data, and a mock mode that needs no cloud.*

Live demo walkthrough

Helix Therapeutics, a fictional pharma company, on synthetic data

DEMO GOAL

Make every agent call identified, policed and traced.

1. Act 1, Chaos: a rogue write succeeds
2. Act 2, Identity and Toolbox go on
3. Act 3, Policy blocks and pauses calls
4. Act 4, Console and end-to-end trace

PASTE DEMO SCREENSHOT
OR PRODUCT VIEW HERE

Fallback: DEMO_MODE=mock runs all four acts offline, same governance code

Five things to do on Monday

The order matters. You cannot police agents you have not listed.

01	**Inventory**	List every agent, its owner, and every tool it can call
02	**Identity**	Move agents off shared keys onto their own Entra identities
03	**Tools**	Put tools behind a versioned Toolbox endpoint, not direct wiring
04	**Policy**	Deny by default. Every write needs an explicit allow or a human
05	**Trace**	Emit OpenTelemetry with the policy decision on the span

What's GA and what's preview

The slide vendor demos skip. Re-check every row before you build on it.

01

01

Agent Framework

Version 1.0 is GA since April 2026. Stable APIs and long-term support. Build on it.

02

02

Foundry Toolboxes

Public preview. Pilot it, pin your versions, and expect the surface to change.

03

03

A2A adapter

Beta at the 1.0 launch. Wrap it behind your own gateway interface.

04

04

Entra Agent ID

Moving fast. Verify the current status in the week before you present.

The pattern is the product. Put an interface around anything in preview so you can swap it.

Continue learning

Everything from the demo runs on your laptop in mock mode

SLIDES	Session deck and speaker notes	[add link]
CODE	Demo repo: governance layer, four acts	[add repo link]
DOCS	Microsoft Learn: Foundry agents	learn.microsoft.com
COMMUNITY	MMAUG events and updates	mmaug.com

ADD QR CODE

Points at the demo repo

Meetup: meetup.com/malta-microsoft-ai-user-group

THANK YOU.

> *Governance is what lets you ship agent number 21.*

WEBSITE
mmaug.com

MEETUP
Malta Microsoft AI User Group

SPEAKER
rajanieshkaushikk.com